

[1 Обязательное задание](#)[2 Варианты](#)[3 Нормы](#)[4 Вопросы для самопроверки](#)[5 Факультативное задание](#)

# Лабораторная работа 5. Множества и словари

## 1 Обязательное задание

Реализовать лингвистический анализ литературного произведения на английском языке. Для примера лингвистического анализа см., например, [Лев Толстой. Война и мир. Том первый](#). В данной работе в рамках лингвистического анализа нужно определить следующие характеристики текста:

- количество уникальных слов в тексте;
- слово, которое встречается в тексте наиболее часто, и количество его вхождений.

### Требования ко всем с лабораторным работам

1. Скачать текстовый файл с литературным произведением согласно номеру варианта.

- Найти по названию страницу литературного произведения на [Project Gutenberg](#).
- На странице выбрать `Plain Text UTF-8` для скачивания нужного файла с текстом литературного произведения.
- Открыть скачанный файл в текстовом редакторе, проверить корректность его содержимого и удалить метаинформацию в начале и в конце текста при её наличии.

2. Написать функцию для загрузки содержимого файла в память в виде вектора строк.

В качестве параметра функции предусмотреть путь к файлу в виде `std::string` или `std::filesystem::path`.

Функция должна вернуть вектор строк (набор отдельных слов во входном файле).

Например, чтение строк (объектов типа `std::string`) с помощью последовательного вызова оператора `<<` для следующего входного текста

```
_In the first place, the book (it may be barely necessary to remind the read
```

приведёт к следующему набору считанных строк:

```
{ "_In", "the", "first", "place,", "the", "book", "(it", "may", "be", "barely
```

Для унификации представления одних и тех же слов полученный набор должен быть нормализован, например, следующим образом:

```
"_In"      → "in"  
"place, "  → "place"  
"(it"      → "it"  
"reader)"  → "reader"  
"early, "  → "early"  
"1796, "   → "" (игнорируется)  
"... "     → "" (игнорируется)
```

### Рекомендуемый способ нормализации слова

1. Привести все буквы к нижнему регистру (см. [std::tolower](#)).
2. Удалить все символы, не являющиеся буквами английского алфавита (см. [std::isalpha](#)), в начале и конце слова.
3. Слово считается прошедшим нормализацию, если:
  - остался хотя бы один символ;
  - все имеющиеся символы — это латинские буквы или дефисы;
  - первый и последний символ не являются дефисом;
  - два дефиса не стоят подряд.

В выходной вектор добавляются только слова, прошедшие нормализацию.

Нормализацию слова оформить в виде отдельной вспомогательной функции. Если слово не удалось нормализовать, вспомогательная функция может, например, вернуть пустую строку, которая не должна добавляться в выходной вектор.

3. Написать функцию для подсчёта уникальных слов в векторе слов. Во время подсчёта не учитывать незначимые слова (к ним будут относиться артикли, предлоги, местоимения и т. п.). Для подсчёта использовать неупорядоченное множество.

Параметры функции:

- вектор слов;
- неупорядоченное множество незначимых слов.

Возвращаемое значение:

- количество уникальных слов.

4. Написать функцию для нахождения наиболее часто встречающегося слова и количества его вхождений. Если таких слов несколько, найти их все. Во время подсчёта не учитывать незначимые слова. Для решения задачи использовать неупорядоченный словарь.

Параметры функции:

- вектор слов;
- неупорядоченное множество незначимых слов.

Возвращаемые значения:

- Количество вхождений наиболее часто встречающегося слова (слов).
- Упорядоченный набор этих слов. Набор слов может быть:
  - отсортированным вектором строк (см. `std::sort`);
  - упорядоченным множеством (см. `std::set`).

Примеры {входа} → {выхода} такой функции в предположении, что набор незначимых слов пуст:

- `{ }` → `0, { }`;
- `{ "mother", "father" }` → `1, { "father", "mother" }`;
- `{ "mother", "father", "son", "daughter", "son", "son" }` → `3, { "son" }`;
- `{ "mother", "father", "son", "daughter", "daughter", "son" }` → `2, { "daughter", "son" }`.

5. Написать функцию `main`, в которой:

- Загрузить литературное произведение из текстового файла, подготовленного в пункте 1, в вектор слов, используя функцию из пункта 2.

- Создать неупорядоченное множество незначимых слов с высокой частотой повторяемости (артикли, предлоги, местоимения и т. п.).
- Посчитать и вывести количество уникальных слов в произведении (за исключением незначимых слов) с помощью функции из пункта 3.
- Вывести наиболее часто встречающееся слово (за исключением незначимых слов) и количество его вхождений. Если таких слов несколько, вывести их все. Использовать функцию из пункта 4.

6. Все созданные функции должны быть вынесены в отдельный модуль.

Чек-лист в виде требуемого набора функций (как минимум):

- функция для загрузки текстового файла в вектор строк;
  - вспомогательная функция для нормализации слова;
- функция для подсчёта уникальных слов в векторе слов;
- функция для нахождения наиболее часто встречающегося слова и определения количества его вхождений.

Пусть анализируемый текстовый файл `pride_and_prejudice_by_jane_austen.txt` содержит следующий текст:

Walt Whitman has somewhere a fine and just distinction between "loving by allowe

Тогда пример возможного вывода выглядит следующим образом:

-----

ЛР №: 5

Группа: 6111

Автор: Иванов Иван

Вариант: 99

-----

Анализ текста из файла:

"pride\_and\_prejudice\_by\_jane\_austen.txt"

Общее количество слов в тексте: 18.

Незначимые слова:

"a" "and" "between" "by" "has" "just" "with"

Количество уникальных слов: 9

Количество вхождений самого популярного слова: 2

Самые популярные слова:

"loving"

## 2 Варианты

1. A Journey to the Centre of the Earth by Jules Verne
2. Adventures of Huckleberry Finn by Mark Twain
3. Alice's Adventures in Wonderland by Lewis Carroll
4. Anna Karenina by graf Leo Tolstoy
5. Around the World in Eighty Days by Jules Verne
6. Dracula by Bram Stoker
7. Fairy Tales by Hans Christian Andersen
8. Frankenstein; Or, The Modern Prometheus by Mary Wollstonecraft Shelley
9. Grimms' Fairy Tales by Jacob Grimm and Wilhelm Grimm
10. Gulliver's Travels into Several Remote Nations of the World by Jonathan Swift
11. King Solomon's Mines by H. Rider Haggard
12. Macbeth by William Shakespeare
13. Peter Pan by J. M. Barrie
14. Romeo and Juliet by William Shakespeare
15. The Adventures of Sherlock Holmes by Arthur Conan Doyle
16. The Adventures of Tom Sawyer by Mark Twain
17. The Art of War by Sunzi
18. The Brothers Karamazov by Fyodor Dostoyevsky
19. The Call of the Wild by Jack London
20. The Iliad by Homer
21. The Life and Adventures of Robinson Crusoe by Daniel Defoe
22. The Odyssey by Homer
23. The Picture of Dorian Gray by Oscar Wilde
24. The Republic by Plato
25. The Sign of the Four by Arthur Conan Doyle
26. The Sun Also Rises by Ernest Hemingway
27. The Wonderful Wizard of Oz by L. Frank Baum
28. Twenty Thousand Leagues under the Sea by Jules Verne
29. War and Peace by graf Leo Tolstoy
30. White Fang by Jack London

## 3 Нормы

См. раздел [Containers](#).

## 4 Вопросы для самопроверки

- Что такое статическая и динамическая типизация?
- Какой вид типизации используется в C++?
- Что такое контейнер?
- Что такое множество?
- Что такое словарь?
- Что такое итератор?
- Что такое инвалидация итератора?
- Приведите пример возможной инвалидации итератора при работе с вектором.
- Какой наиболее предпочтительный способ вернуть из функции несколько значений (возможно, разного типа)?

## 5 Факультативное задание

Факультативная часть засчитывается только при сдаче лабораторной работы на операционной системе семейства Linux.

Необходимо находить не только самое часто встречающееся слово, а  первых наиболее часто встречающихся слов, где  задаётся пользователем через стандартный поток ввода. Если какие-то слова встречаются одинаковое количество раз, то вывести их все.

Изменение заключается в модификации (улучшении) функции из пункта 4 обязательного задания. У функции появляется дополнительный параметр . При этом возвращаться будет не один набор, а вектор наборов (с количеством наборов в векторе не более ).

### Пример

Пусть анализируемый текст выглядит следующим образом:

```
b b b   a a a a   d d d   c c c   e f
```

Результат работы улучшенной функции из пункта 4 обязательного задания в зависимости от введённого значения :

- `n = 1`:

4: "a"

- Наибольшее количество раз (4) встречается только одно слово `a`.

- `n = 2`:

4: "a"

3: "b" "c" "d"

- Два наиболее часто встречающихся слова входят в текст в количестве 4 и 3 штук. 4 раза встречается слово `a`, 3 раза встречаются слова `b`, `c` и `d`.

- `n >= 3`:

4: "a"

3: "b" "c" "d"

1: "e" "f"